1 · NUMBER SYSTEMS

Bin	Hex	Dec
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Convert	Group
Bin→Oct	3 bits, from right
Bin→Hex	4 bits, from right
Hex→Bin	digit → 4 bits
Pad	0s on the left

2^n	value
2^4	16
2^8	256
2^{10}	1024
2^{16}	65536
2^{20}	1 M
2^{32}	4 G

Base conversion / arithmetic

Dec→base b: divide by b repeatedly; read remainders bottom→top.

Fraction→base b: multiply by b repeatedly; read integer parts top→bottom.

Base b→Dec: Σ digit·b^{pos} (fraction: b^{-1}, b^{-2}…).

Add in base b: digit-wise; if sum ≥ b write sum−b, carry 1.

Subtract: if top < bottom, borrow adds **b** (base 5: +5) to that digit.

n-bit scheme	Range	Negate / notes
Unsigned	0 … 2^n-1	—
Sign-magnitude	$\pm(2^{n-1}-1)$	flip MSB; two zeros
1's complement	$\pm(2^{n-1}-1)$	flip all bits; two zeros
2's complement	**-2^{n-1} … $2^{n-1}-1$**	**invert all bits, +1**; one zero
Excess-K	stored = value+K	value = stored − K

2's-complement rules

MSB=1 ⇒ negative. Value = $-MSB\cdot2^{n-1}$ + rest.

Sign-extend = copy MSB leftwards (value unchanged): 0xF5 (8b) → 0xFFF5; 0x75 → 0x0075.

Overflow (add): (+)+(+)→neg, or (−)+(−)→pos. Carry-in to MSB ≠ carry-out of MSB. (+)+(−) never overflows.

Subtract: A−B = A + (~B) + 1.

Shift left k = ×2^k. Arithmetic shift right k = ÷2^k (keeps sign); logical shift right fills 0.

Dec→2's comp: −18 → 00010010 → invert 11101101 → +1 = **11101110** = 0xEE.

All-ones n-bit = −1 (0xFF, 0xFFFFFFFF).

5b · Encode → hex recipe

1. Fill every field in **binary** (reg no. → 5 bits; neg imm → 16-bit 2's comp).

2. Concatenate 32 bits. 3. Group in 4 bits → hex (prefix 0x).

beq/bne offset = number of instructions from the instruction **after** the branch (PC+4) to the target. Forward +, backward −. Target address = PC+4 + 4·offset.

j/jal field = (target address ÷ 4), keep low 26 bits. Full target = PC+4[31:28] ‖ field ‖ 00.

Finding addresses: addr(instr n) = addr(first instr) + 4·n. Pseudo-instructions that expand to 2 real ones shift all later addresses!

Decode: top 6 bits=0 → R-type (read funct). Else op gives the instruction.

Reg numbers: $v0-1=2-3, $a0-3=4-7, $t0-7=8-15, $s0-7=16-23, $t8-9=24-25, $sp=29, $ra=31.

1b · IEEE-754 FLOATING POINT

Value = $(-1)^S \times 1.F \times 2^{E-bias}$ (bias: 127 single, 1023 double)

	S	Exponent E (bias 127)	Fraction F
Single	1	8	23

	S	Exp E (1023)	Fraction F
Double	1	11	52

E	F	Meaning
0	0	±0
0	≠ 0	denormal: 0.F × 2^{-126}
1 … 254	any	normal: 1.F × 2^(E−127)
255	0	±∞
255	≠ 0	NaN

Encode decimal → IEEE hex (steps)

① Sign bit S (1 if negative). Work with |x|.

② Integer part → binary (÷2). Fraction part → binary (×2, take integer part).

③ Normalise: move point so form is 1.xxx × 2^k (k = places moved; left = +).

④ E = k + 127 → 8-bit binary.

⑤ F = bits after the leading 1; pad zeros on the right to 23 bits.

⑥ Write S|E|F (1+8+23), regroup in 4 bits from the left → hex.

Decode: hex → bits → S,E,F → $(-1)^S\cdot1.F\cdot2^{(E-127)}$.

Fraction n/2^m → write n in m bits (0.1484375 = 19/128 = .0010011).

Checks: 1.0 = 0x3F800000 ; 2.0 = 0x40000000 ; −1.0 = 0xBF800000 ; 0 = 0x00000000.

Caveats: 0.1, 0.2 not exact (repeating binary) → rounding. Big + small loses small. Only 24 significant bits.

2 · MIPS BASICS

#	Name	Use	Saved?
0	$zero	constant 0 (writes ignored)	—
1	$at	assembler temp (pseudo)	—
2-3	$v0-$v1	return values	no
4-7	$a0-$a3	arguments	no
8-15	$t0-$t7	temporaries	no
16-23	$s0-$s7	saved	**yes**
24-25	$t8-$t9	temporaries	no
26-27	$k0-$k1	kernel	—
28	$gp	global pointer	yes
29	$sp	stack pointer	yes
30	$fp	frame pointer	yes
31	$ra	return address (jal)	yes

Memory & architecture facts

32 registers × 32 bits. Instruction = 32 bits (4 bytes). PC += 4.

Memory is **byte-addressed**; a word = 4 bytes and must be **aligned** (address multiple of 4).

Array int A[i] at base + 4·i. Next word = +4.

Load-store architecture: only lw/sw/lb/sb touch memory; ALU ops use registers/immediates.

Logic: AND with 1 keeps bit / with 0 clears; OR with 1 sets; XOR with 1 flips. x&(2^k−1) = x mod 2^k.

Worked examples (generic)	Hex
add $t0,$s1,$s2 000000 10001 10010 01000 00000 100000	0x02324020
lw $t0,8($s1) 100011 10001 01000 0000000000001000	0x8E280008
beq $t0,$zero,+2 instr 000100 01000 00000 0000000000000010	0x11000002
sll $t1,$t0,2 000000 00000 01000 01001 00010 000000	0x00084880
j 0x00400020 → field 0x100008 000010 00000100000000000000001000	0x08100008

3 · REAL MIPS INSTRUCTIONS + FORMAT (fields in binary; hex = op/funct)

Instr	Usage	What it does	op	rs	rt	rd	shamt	funct	hex
R-type	op=000000: fields		6	5	5	5	5	6	
add	add $d,$s,$t	$d = $s + $t (traps on overflow)	000000	$s	$t	$d	00000	100000	0/20
addu	addu $d,$s,$t	$d = $s + $t (no trap)	000000	$s	$t	$d	00000	100001	0/21
sub	sub $d,$s,$t	$d = $s − $t (traps on overflow)	000000	$s	$t	$d	00000	100010	0/22
subu	subu $d,$s,$t	$d = $s − $t (no trap)	000000	$s	$t	$d	00000	100011	0/23
and	and $d,$s,$t	$d = $s AND $t	000000	$s	$t	$d	00000	100100	0/24
or	or $d,$s,$t	$d = $s OR $t	000000	$s	$t	$d	00000	100101	0/25
xor	xor $d,$s,$t	$d = $s XOR $t	000000	$s	$t	$d	00000	100110	0/26
nor	nor $d,$s,$t	$d = NOT($s OR $t)	000000	$s	$t	$d	00000	100111	0/27
slt	slt $d,$s,$t	$d = 1 if $s < $t else 0 (signed)	000000	$s	$t	$d	00000	101010	0/2A
sltu	sltu $d,$s,$t	same, unsigned compare	000000	$s	$t	$d	00000	101011	0/2B
sll	sll $d,$t,sh	$d = $t << sh (rs unused = 0)	000000	00000	$t	$d	sh	000000	0/00
srl	srl $d,$t,sh	$d = $t >> sh, zero fill	000000	00000	$t	$d	sh	000010	0/02
sra	sra $d,$t,sh	$d = $t >> sh, sign fill	000000	00000	$t	$d	sh	000011	0/03
jr	jr $s	PC = $s (return)	000000	$s	00000	00000	00000	001000	0/08
I-type	imm = 16-bit (sign-ext; andi/ori/xori zero-ext)		6	5	5	16 bits			
addi	addi $t,$s,imm	$t = $s + SignExt(imm)	001000	$s	$t	imm16			8
addiu	addiu $t,$s,imm	same, no overflow trap	001001	$s	$t	imm16			9
slti	slti $t,$s,imm	$t = 1 if $s < SignExt(imm)	001010	$s	$t	imm16			A
sltiu	sltiu $t,$s,imm	unsigned compare	001011	$s	$t	imm16			B
andi	andi $t,$s,imm	$t = $s AND ZeroExt(imm)	001100	$s	$t	imm16			C
ori	ori $t,$s,imm	$t = $s OR ZeroExt(imm)	001101	$s	$t	imm16			D
xori	xori $t,$s,imm	$t = $s XOR ZeroExt(imm)	001110	$s	$t	imm16			E
lui	lui $t,imm	$t = imm << 16 (low 16 = 0)	001111	00000	$t	imm16			F
lw	lw $t,off($s)	$t = Mem[$s + off] (word)	100011	$s base	$t	off16			23
sw	sw $t,off($s)	Mem[$s + off] = $t	101011	$s base	$t	off16			2B
lb	lb $t,off($s)	load byte, sign-extend	100000	$s base	$t	off16			20
lbu	lbu $t,off($s)	load byte, zero-extend	100100	$s base	$t	off16			24
sb	sb $t,off($s)	store low byte of $t	101000	$s base	$t	off16			28
beq	beq $s,$t,L	if $s = $t: PC = PC+4+4·off	000100	$s	$t	off16 = (L−(PC+4))/4			4
bne	bne $s,$t,L	if $s ≠ $t: branch	000101	$s	$t	off16 = (L−(PC+4))/4			5
J-type	target = word address		6	26 bits					
j	j L	PC = PC+4[31:28] ‖ addr ‖ 00	000010	L >> 2 (low 28 bits of L, drop 2)					2
jal	jal L	$ra = PC+4 ; then jump	000011	L >> 2 (same as j)					3

† only if taught. R-type asm order is **rd, rs, rt** but binary order is **rs, rt, rd**. Shifts: rs=0, source in rt, amount in shamt. lw/sw: rs=base reg, rt=data reg. Branch/jump: no rd.

5 · ENCODING / DECODING

R-type	**op** 6	**rs** 5	**rt** 5	**rd** 5	**shamt** 5	**funct** 6
I-type	**op** 6	**rs** 5	**rt** 5	**immediate / offset** 16		
J-type	**op** 6	**target address (26 bits)** 26				

Bit positions: op [31-26], rs [25-21], rt [20-16], rd [15-11], shamt [10-6], funct [5-0]; imm [15-0]; addr [25-0].

Addressing mode	Used by	Address / operand
Register	add, sub, and, slt…	operands in rs, rt; result rd
Immediate	addi, andi, ori, slti, lui	operand = imm16 (sign- or zero-extended)
Base + offset	lw, sw, lb, sb	Mem[reg[rs] + SignExt(off16)]
PC-relative	beq, bne	target = (PC+4) + SignExt(off16) × 4
Pseudo-direct	j, jal	target = PC+4[31:28] ‖ addr26 ‖ 00
Register (jump)	jr	target = reg[rs]

4 · PSEUDO-INSTRUCTIONS (each real instr's format shown)

Pseudo	Meaning	Real instr(s)	op	rs	rt	rd	shamt	funct
			6	5	5	5	5	6
move $d,$s	$d = $s	add $d,$zero,$s	000000	00000	$s	$d	00000	100000
li $t,imm16	$t = const	addi $t,$zero,imm	001000	00000	$t	imm16		
li $t,imm32	32-bit const	lui $t,hi16	001111	00000	$t	hi16		
		ori $t,$t,lo16	001101	$t	$t	lo16		
la $t,label	address	lui $t,hi16	001111	00000	$t	hi16		
		ori $t,$t,lo16	001101	$t	$t	lo16		
subi $t,$s,k	$t = $s − k	addi $t,$s,−k	001000	$s	$t	−k (2's comp)		
not $d,$s	$d = ~$s	nor $d,$s,$zero	000000	$s	00000	$d	00000	100111
neg $d,$s	$d = −$s	sub $d,$zero,$s	000000	00000	$s	$d	00000	100010
nop	do nothing	sll $0,$0,0	000000	00000	00000	00000	00000	000000
blt $s,$t,L	$s < $t ?	slt $at,$s,$t	000000	$s	$t	$at	00000	101010
		bne $at,$zero,L	000101	$at	00000	off		
bgt $s,$t,L	$s > $t ?	slt $at,$t,$s	000000	$t	$s	$at	00000	101010
		bne $at,$zero,L	000101	$at	00000	off		
ble $s,$t,L	$s ≤ $t ?	slt $at,$t,$s	000000	$t	$s	$at	00000	101010
		beq $at,$zero,L	000100	$at	00000	off		
bge $s,$t,L	$s ≥ $t ?	slt $at,$s,$t	000000	$s	$t	$at	00000	101010
		beq $at,$zero,L	000100	$at	00000	off		
beqz $s,L	$s == 0 ?	beq $s,$zero,L	000100	$s	00000	off		
bnez $s,L	$s != 0 ?	bne $s,$zero,L	000101	$s	00000	off		

Caveat: 1 pseudo line may be 2 real instructions (li/la 32-bit, blt/bgt/ble/bge) ⇒ changes instruction count & addresses. "Pseudo not allowed" ⇒ write slt+bne/beq yourself. $at = $1 is reserved for this. No real subi/move/li/la/blt/bgt/ble/bge/not/neg.

6 · C → MIPS PATTERNS

```
# if (a==b) {X} else {Y}    a=$s0 b=$s1
 bne $s0,$s1,Else
 X ...
 j   End
Else: Y ...
End:
```

```
# while (i<n) { ..; i++ }    i=$t0 n=$t1
Loop: slt $t2,$t0,$t1   # i<n ?
 beq $t2,$zero,Exit
 ...
 addi $t0,$t0,1
 j   Loop
Exit:
```

```
# do { } while (x<50): test at bottom
Loop: ... body ...
 slti $t2,$t0,50
 bne  $t2,$zero,Loop
```

```
# x%4==0 : power-of-2 mask
 andi $t1,$t0,3     # x &amp;amp; (4-1)
 beq  $t1,$zero,Yes
# odd? andi $t,$x,1
# x*2^k: sll ; x/2^k: srl/sra
```

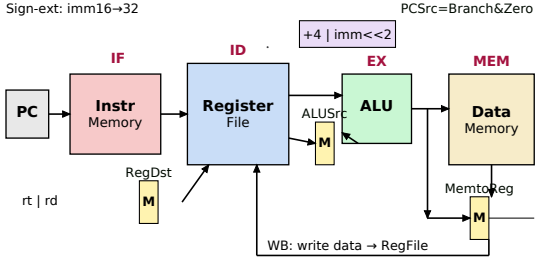
```
# variable at fixed address (no li/la)
 lui $t0,0x3FCD
 ori $t0,$t0,0x0040   # t0=0x3FCD0040
 lw  $s0,0($t0)       # load ONCE
 ... loop works on $s0 ...
 sw  $s0,0($t0)       # store ONCE
```

```
# A[i]  (base $a0, i in $t0)
 sll $t1,$t0,2        # i*4
 add $t1,$t1,$a0
 lw  $t2,0($t1)
# or walk a pointer: addi $a0,$a0,4
```

```
# function call / stack
Func: addi $sp,$sp,-8   # push
 sw  $ra,4($sp)
 sw  $s0,0($sp)
 ... result in $v0
 lw  $s0,0($sp)    # pop
 lw  $ra,4($sp)
 addi $sp,$sp,8
 jr  $ra           # caller: jal Func
```

Exam tips • Keep variables in registers; minimise lw/sw (load once, store once). • Comment every line. • Immediate is only 16 bits: bigger constants need lui + ori. • addi imm −32768...32767; andi/ori zero-extended 0...65535. • **Counting executed instructions**: setup + (instructions per iteration × iterations) + final failing test; instructions skipped by a taken branch are not counted; branches and j count as instructions. Trace loops in a table: iteration | register values. • $zero can never be changed. • Odd/even or divisible-by-2^k tests: use andi mask.

7 · DATAPATH & CONTROL



Stage	Work	Used by
IF	fetch Instr[PC]; PC+4	all
ID	decode; read regs; sign-extend	all
EX	ALU op / address / compare	R, lw, sw, beq
MEM	data memory read/write	**lw, sw only**
WB	write result to register	R, lw (not sw, beq)

Instr	Reg Dst	ALU Src	Mem toReg	Reg Wr	Mem Rd	Mem Wr	Br	ALU Op
R-type	1	0	0	1	0	0	0	10
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00
beq	X	0	X	0	0	0	1	01
addi	0	1	0	1	0	0	0	00

X = don't care. ALUOp: **00 add, 01 subtract, 10 look at funct**.

MUX	sel = 0	sel = 1
RegDst	rt [20-16]	rd [15-11]
ALUSrc	RD2 (register)	sign-ext imm
MemtoReg	ALU result	memory data
PCSrc	PC+4	branch target

PCSrc = Branch AND Zero. Branch target = (PC+4) + SignExt(imm)≪2. Jump = PC+4[31:28] ‖ addr ‖ 00.

ALUOp	funct	ALU ctrl	op
00	xxxxxx	0010	add (lw/sw/addi)
01	xxxxxx	0110	sub (beq)
10	100000	0010	add
10	100010	0110	sub
10	100100	0000	AND
10	100101	0001	OR
10	101010	0111	slt

Mis-wired datapath questions: for each instruction ask "which mux input does it now use?". Example: ALUSrc swapped ⇒ the ALU's 2nd input is RD2 for I-type (addi uses reg[rt] instead of imm) and the immediate field bits (low 16 bits of the instruction) for R-type. Result still goes to the normal destination. Look up register values from the table given. To find a "fix" instruction, pick one whose result is the same either way (e.g. rt value = imm).

Single-cycle vs multi-cycle timing

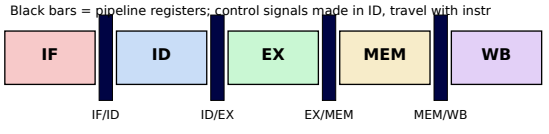
Single-cycle: cycle time = sum of all stage times used by the slowest instr (lw: IF+ID+EX+MEM+WB). CPI = 1. Clock rate = 1/cycle time.

Units: 1 ns period → 1000 MHz. f(MHz) = 1000 / T(ns). Cycle time = 1/f (500 MHz → 2 ns; 100 MHz → 10 ns).

Multi-cycle: cycle = longest stage; CPI by type: R=4, lw=5, sw=4, beq=3, j=2.

8 · PIPELINING & PERFORMANCE

5 stages (N=5): **IF → ID → EX → MEM → WB**



	1	2	3	4	5	6	7
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB

Columns = clock cycles. First instr finishes at cycle N; then one per cycle ⇒ I+N−1 cycles.

Type	Cycle time	Cycles	Total time
Single-cycle	ΣT_k	I (CPI=1)	I × ΣT_k
Multi-cycle	max T_k	I × CPIavg	I × CPIavg × max T_k
Pipelined	max T_k + Tov	I + N − 1	(I+N−1) × (max T_k + Tov)

T_k = time of stage k; Tov = pipeline-register overhead; CPIavg = Σ (fraction of instr × its CPI).

Derivation of speed-up

① Non-pipelined time = I × ΣT_k

② Pipelined time = (I+N−1) × (max T_k + Tov)

③ Speedup = ① ÷ ② = **I·ΣT_k / [(I+N−1)(max T_k + Tov)]**

④ Ideal case: equal stages T_k = T (so ΣT_k = N·T), Tov = 0: speedup = I·N·T / [(I+N−1)·T] = **I·N / (I+N−1)**

⑤ If I ≫ N, (I+N−1) ≈ I ⇒ **speedup → N** (number of stages).

⑥ Throughput = 1 instr per cycle = 1/(max T_k + Tov). One instr's latency is NOT reduced (still N cycles).

Example: stages 2,1,2,2,1 ns, Tov 0.5 ns, I=100 ⇒ cycle 2.5 ns, time = (100+5−1)×2.5 = **260 ns**. Single-cycle: 100 × 8 = 800 ns.

Pipeline facts & caveats

Gains **throughput**, not latency. Rate limited by **slowest stage + overhead**.

Speed-up < N if stages unequal, overhead > 0, or stalls.

Pipeline registers: IF/ID, ID/EX, EX/MEM, MEM/WB (none after WB).

Control signals by stage — EX: RegDst, ALUOp, ALUSrc · MEM: Branch, MemRead, MemWrite · WB: MemtoReg, RegWrite. Generated in ID, carried along.

Write-register number must be carried ID/EX → EX/MEM → MEM/WB; otherwise WB uses a later instruction's field (the corrected datapath).

IF/ID holds instr and PC+4; ID/EX holds reg values, sign-ext imm, PC+4.

Dependency (next instr needs result not yet written) ⇒ **stall**/bubble: cycles = I+N−1+stalls.

In diagrams, draw a staircase; count up to the last instr's WB.

Performance basics

Exec time = Instruction count × CPI × cycle time. Speedup = old time ÷ new time.

Clock rate = 1 / cycle time. Throughput = instr per second.

9 · LAST-MINUTE CHECKLIST

☑ ns→MHz: 1000/ns. ☑ Hex answers: write with 0x. ☑ addi sign-extends, andi/ori zero-extend. ☑ Shifts: rs=0, shamt in bits 10-6. ☑ lw/sw: rs=base. ☑ Branch offset counted from PC+4 in words. ☑ Pseudo may be 2 instr. ☑ Show working.